Repository: https://github.com/Manjunath-G-K/isocode-engineering-challenge

SHIPMENTIQ

Technical Design Document

A traceable, human-in-the-loop customs documentation workbench for shipment intake, extraction, cross-document review, and controlled provisional export.

Evaluation boundary

This document describes the implementation in this repository. It does not claim production hosting, legal compliance, filing-portal submission, a real client import-template match, extraction accuracy, or performance targets.

Parts 1-4	0001-0006	PostgreSQL	Human review
Implemented scope	Alembic migrations	System of record	Approval boundary

Submission contents

ShipmentIQ is implemented as a React/Vite client and a FastAPI modular monolith. The backend owns authentication, authorization, extraction state, review decisions, validation, stale-state handling, and CSV eligibility. The frontend renders those decisions and calls real APIs.

- Shipment management: create, list, detail, metadata update, server-controlled statuses, and object-level access.
- Document workflow: private PDF/JPG/PNG upload, authorized viewer/download, multi-page OCR, provenance, and persisted processing status.
- Review workflow: confidence and source evidence, field correction, confirmation, rejection, revision history, discrepancy flags, and manager reporting.
- Export workflow: backend-recomputed eligibility and a clearly provisional CSV layout; unresolved or stale work remains blocked.

01

Product boundary and architecture

ShipmentIQ is a documentation control tower for import shipments. It stores a shipment and its source documents, proposes structured values, makes uncertainty visible, supports human review, compares current values across available documents, and prepares a backend-gated CSV response.

In scope

- React 18, TypeScript, Vite, and plain CSS for sign-in, shipment listing, shipment creation, shipment detail, document viewing, field review, validation, export controls, revision history, and manager reporting.
- FastAPI, SQLAlchemy 2, Pydantic 2, Alembic, and PostgreSQL for the trusted application boundary.
- Provider-neutral OCR and extraction interfaces with a free local default and an explicit optional OpenAI-compatible adapter.
- Private generated-name local document storage behind a replaceable storage protocol.

Out of scope by design

- No customs portal submission, filing connector, legal certification, or claim of portal acceptance.
- No HSN classification or tariff lookup; HSN is only extracted when written in a source document.
- No duty calculation, importer tracking portal, CSV export based on an invented client template, or real production deployment.
- No Redis, Kafka, worker service, or microservice split; synchronous processing keeps the local workflow observable and small.

System shape

Browser

React/Vite UI sends credentialed REST requests. The browser receives a session cookie and a readable CSRF cookie after login; state-changing calls send the matching configured header.

FastAPI modular monolith

Authentication/RBAC, shipments, documents/storage, extraction, review/validation, exports, reports, health, and consistent error handling share one transaction boundary.

PostgreSQL + private storage

PostgreSQL stores users, shipments, document metadata, extraction results, review fields, revisions, and validation results. Document bytes remain in private local storage; the storage adapter is replaceable.

Why a modular monolith

- Authorization, review state, validation state, and export eligibility remain close to the transaction that changes them.
- Synchronous extraction makes OCR/AI failures and statuses visible without adding a queue or worker failure mode before provider latency is known.
- A provider protocol and service boundaries preserve replaceability without paying the operational cost of distributed deployment.

02

Technology choices and data model

Area	Implemented choice	Reason and boundary
Frontend	React 18 + TypeScript + Vite + plain CSS	No UI framework or client-side state dependency; API calls and backend decisions remain explicit.
API	FastAPI + Pydantic v2	Typed request/response contracts and consistent JSON error bodies with request IDs.
Persistence	SQLAlchemy 2 + PostgreSQL	Transactional state, row locks for export snapshots, and database-backed RBAC.
Migrations	Alembic 0001 through 0006	Reproducible schema evolution from foundation through review/validation.
Storage	PrivateFileStorage behind DocumentStorage	Generated names, size/content checks, and no public document path.
Security	Argon2id + JWT HttpOnly cookie + signed CSRF cookie/header	Short-lived session, server-side role reload, and explicit origin/CSRF checks.

Persisted model

Table	Purpose	Important behavior
users / roles / user_roles	Active identities and roles	Roles are reloaded from PostgreSQL for each protected request; JWT role claims are not trusted.
shipments	Shipment record and lifecycle	Unique normalized reference, importer/supplier/origin, creator, timestamps, and backend-controlled status.
shipment_documents	Document metadata	One invoice, packing list, or bill of lading per type; checksum, content type, size, and private storage key.
document_extractions	Latest processing result	Provider names, status, page count, JSON result, timestamps, and safe failure code.
review_fields	Flattened reviewable values	Field path, extracted/current value, type, confidence, source references, status, and optimistic version.
field_revisions	Append-only review history	Actor, action, previous/new value, note, and timestamp.
shipment_validations	Current deterministic checks	Rule key, evidence JSON, status, evaluator, time, and STALE invalidation.

Migration state

The schema is applied through migration 0006_review_validation. Part 3 CSV export is streamed from current state and does not add a separate export-attempt table.

03

Trust boundaries and end-to-end workflows

Authentication, CSRF, and authorization

- POST /api/v1/auth/login validates the request, authenticates the normalized email and Argon2id password, then sets a short-lived HttpOnly session cookie and a readable signed CSRF cookie.
- State-changing requests require an allowed Origin and a matching CSRF cookie/header pair bound to the session token. Cookie names and the CSRF header name are environment-configurable.
- Local CORS allows the configured localhost origins with credentials. The frontend default API host follows the browser's localhost or 127.0.0.1 hostname so host-only CSRF cookies remain readable.
- Protected routes reload the active user and roles from PostgreSQL. Shipment, document, field, revision, validation, report, and export access is checked in backend services.

Document intake and access

- An executive creates a DRAFT shipment, declares a document type, and uploads a PDF, JPG, or PNG within the configured size limit.
- The service checks extension, declared type, file signature, basic document structure, size, and one-document-per-type rules before private storage.
- The viewer calls the authorized download endpoint; it never receives a public storage path. Managers can review authorized shipments but cannot upload.

State sequence

Step	Command	Trusted outcome
1. Intake	Create shipment and upload document	Server validates role, object scope, document bytes, and status transition.
2. Extract	Run OCR and structured provider	Persist pages, source references, confidence, result/status, or safe failure code.
3. Review	Correct, confirm, or reject field	Persist current value, actor/timestamp revision, version increment, and flags.
4. Validate	Run deterministic checks	Persist PASSED, FAILED, or UNRESOLVED evidence using current values.
5. Export	Request provisional CSV	Recompute eligibility under a shipment row lock; block unless all gates pass.

04

OCR, structured extraction, and uncertainty

The extraction boundary deliberately separates reading from approval. OCR and the AI-compatible provider only propose values. Review and validation services decide whether data is ready for a downstream export; extraction itself never approves a shipment.

Replaceable provider contracts

Interface	Current implementation	Failure behavior
OCRProvider	LocalOCRProvider uses pypdf for embedded PDF text and Tesseract through PyMuPDF/Pillow for scanned PDF pages and JPG/PNG.	Unsupported, unreadable, or unavailable OCR dependencies become an OCR failure or unreadable page state.
AIExtractionProvider	LocalRuleBasedProvider is the no-cost default and only accepts explicitly labelled values. OpenAICompatibleProvider is opt-in through environment configuration.	Provider transport or response problems become a safe persisted AI/provider failure.
Storage adapter	DocumentStorage protocol with private local implementation.	Storage validation/access errors are returned as safe API failures; bytes are not exposed publicly.

Supported structured fields

- Parties: importer, supplier, consignee, and exporter when explicitly present.
- Invoice: number, date, currency, Incoterm, and total.
- Line items: description, quantity, unit price, and HSN code as written in the document.
- Shipment details: packages, gross weight, net weight, and containers.
- Provenance: ordered OCR pages with readable/source labels plus field-level page and locator references.

Strict validation and missing data

- Pydantic models use extra=forbid, bounded confidence values, validated document types, ordered unique pages, and the rule that a non-null value requires confidence and source references.
- JSON-native provider output is normalized only for explicit date/decimal/enum conversion before strict validation. A malformed response is not coerced into a plausible value.
- Missing values remain null/empty with no fabricated source. Successful extraction with missing or unreadable values becomes NEEDS_REVIEW; malformed/provider/processing failures are persisted with safe codes.
- A configured paid AI path is never required for the default local workflow. API keys are read from environment configuration only and are not committed.

Interpretation of confidence

A confidence score expresses provider uncertainty and drives review flags. It is not a correctness guarantee, an approval, or evidence that a document is legally valid.

05

Human review, validation, and export control

Human review

- The review API returns the authorized document, extraction status/page count, flattened fields, current and extracted values, confidence, source references, and flags.
- Executives can correct fields or manually enter values for unreadable/missing data. Confirm and reject actions are explicit human decisions; rejected values are cleared instead of replaced with a guess.
- Every correction/decision appends a field revision with actor and timestamp and uses expected_version optimistic concurrency. A stale version is rejected rather than silently overwritten.
- Managers can inspect the workflow, run validations, confirm/reject under the current role policy, and view field/supplier reporting. Server-side RBAC remains authoritative even if the UI hides a button.

Deterministic validation

Rule	Behavior	Uncertainty policy
Invoice total vs. line items	Compares invoice total with the normalized sum of invoice line quantities multiplied by unit prices.	No supplied rounding tolerance is invented; exact normalized values are used.
Package count	Compares package values across available invoice, packing list, and bill of lading fields.	Fewer than two comparable values remains UNRESOLVED.
Gross/net weight	Compares current reviewed weight fields across available documents.	A mismatch is visible, but the service never chooses a correct document.

Stale validation rule

Any correction or rejection marks existing validation results STALE. The reviewer must explicitly rerun validation before an export can become eligible. Missing comparison evidence is unresolved, not a pass.

Backend-enforced provisional CSV

- GET export-eligibility and POST exports both evaluate current state in backend code; the frontend cannot bypass blockers.
- Eligibility requires an invoice, completed/readable extraction, required values, no rejected or low-confidence populated fields, human confirmation, and every deterministic validation rule in PASSED state.
- CSV generation locks the shipment row while recomputing eligibility and taking the snapshot, so concurrent corrections serialize before or after a complete decision.
- The provisional columns are shipment_reference, importer, supplier, origin_country, invoice_number, invoice_date, currency, incoterm, invoice_total, item_description, quantity, unit_price, hsn_code, packages, gross_weight_kg, net_weight_kg, and containers.
- A successful response marks the shipment EXPORTED. Later corrections invalidate review state; there is no separate export-attempt table.

06

UI/UX and role experiences

The UI is intentionally restrained: white/beige paper surfaces, dark green navigation, muted rust/amber status accents, clear field labels, and responsive grids. The client uses plain CSS in src/styles.css and no component library.

Screen	Executive experience	Manager experience
Sign-in	Authenticated login with backend session/CSRF cookies.	Same authenticated login flow.
Shipment list	View owned shipments and create a new shipment.	View authorized shipments and open accuracy reporting.
Shipment detail	Upload, extract, review, correct, confirm/reject, validate, and request export.	View documents/fields, confirm/reject, run validation, inspect blockers and evidence.
Document viewer	Authorized source document beside extracted fields.	Authorized source document beside extracted fields.
Review history	Open field-level revisions after corrections/decisions.	Inspect revision history and report outcomes.
Accuracy report	Not offered as a manager report control.	Field-level and supplier-level review metrics.

Interaction principles

- The original document remains accessible while fields are reviewed; source page and locator evidence sits beside the current value.
- Low confidence, missing, rejected, failed, unresolved, and stale states are rendered as visible badges or blockers.
- Loading, empty, and error states are explicit. Buttons call real backend routes; no fake approval or export workflow exists in the client.
- The frontend is a command/view client. Role visibility improves usability, but authorization and export eligibility are never decided only in the browser.

07

Testing strategy and verification

The project keeps the default unit suite fast and uses an opt-in PostgreSQL integration suite for database-backed behavior. Tests use synthetic data and unique fixture identities; no real client records are required.

Layer	Coverage	Submission verification
Backend default	Configuration, security primitives, storage validation, extraction schemas/providers, review rules, health behavior, export layout, and error handling.	33 passed, 24 skipped, 1 warning.
PostgreSQL integration	Migrations, authentication/RBAC, shipment creation, CSRF rejection, object access, upload races, extraction persistence, review/revisions, stale validation, export gating, and concurrency.	57 passed, 1 warning against configured PostgreSQL.
Frontend	Credentialed API parsing, CSRF cookie/header propagation, backend error propagation, and local-host API base selection.	4 passed.
Production build	TypeScript project build and Vite production bundling.	Passed; CSS and JS assets emitted successfully.

Important test boundaries

- The PostgreSQL integration suite must be run with RUN_DB_INTEGRATION=1 and a valid DATABASE_URL; the default suite intentionally does not substitute SQLite.
- There is no browser-level Playwright suite or production-hosting smoke test in this repository. Frontend API behavior is covered with Vitest/jsdom and the Vite production build.
- A Starlette TestClient deprecation warning is emitted because the pinned stack reports that httpx2 will be required in a future compatibility change; it does not fail the current suite.

Verification commands

Backend default: .\.venv\Scripts\pytest.exe -q

PostgreSQL integration: $env:RUN_DB_INTEGRATION = '1'; .\.venv\Scripts\pytest.exe -q

Frontend: npm run test; npm run build

08

Decisions, assumptions, limitations, and future work

Explicit assumptions

- The default low-confidence threshold is 0.80 and remains configurable because the brief does not supply an operational threshold.
- Exact normalized numeric equality is used for comparisons because no weight or currency tolerance was supplied.
- HSN codes are treated as document-provided strings; no tariff classification is attempted.
- The local labelled-text provider is the no-cost default. A paid or remote provider is used only when explicitly configured.
- The client role policy currently permits manager confirmation/rejection and validation, while corrections/uploads/CSV generation remain executive-only in backend routes.
- SHP-24811 and SHP-24812 are synthetic demonstrations. SHP-24811 demonstrates invoice gross weight 3,120 kg versus packing-list gross weight 3,165 kg; the system reports the discrepancy without selecting the correct source.

Known limitations and security findings

- The actual filing-software Excel/CSV template, column policy, encoding, and required-field rules were not supplied. The current layout is documented as provisional and must not be represented as a filing connector.
- No production object storage, secret manager, HTTPS reverse proxy, backup/restore setup, malware scanning, login rate limiting, refresh-token rotation, or password recovery is implemented.
- Tesseract is an external local prerequisite for scanned-image OCR; the repository does not bundle its executable or language data.
- The optional OpenAI-compatible adapter requires explicit endpoint, key, model, timeout, and a client-approved data-use policy. No provider is configured by default.
- There is no labelled 200-shipment evaluation set, so no field-accuracy, processing-time, amended-entry reduction, cost, or performance figure is claimed.
- India hosting, client-data residency, no-training provider terms, and regulatory/legal acceptance remain deployment and policy decisions, not verified properties of this local repository.

Future improvements when inputs are supplied

- Implement the real client import template and acceptance tests once supplied; preserve the provisional layout until then.
- Confirm document mappings, tolerance/rounding rules, manager approval policy, supplier identity matching, retention, and accuracy formulas with operations.
- Add approved production object storage, secret injection, backups, malware scanning, rate limiting, refresh-token policy, and browser-level end-to-end tests.
- Introduce an asynchronous worker only if real provider latency or volume requires it; keep provider and storage interfaces stable.
- Build a labelled evaluation set and report measured accuracy and review time instead of inferring outcomes from synthetic demos.

09

Submission summary

What is ready

The repository contains the implemented Parts 1-4 workflow: authenticated shipment management, private document intake, OCR/structured extraction with uncertainty, human review, deterministic validation, stale-state handling, backend-gated provisional CSV export, role-based UI, synthetic demo scripts, migrations, tests, and local deployment guidance.

What a reviewer can verify locally

- Start PostgreSQL with the existing Docker Compose file and apply Alembic migrations through 0006_review_validation.
- Set local-only backend environment variables, seed synthetic reviewer accounts and labeled shipment data, and start the FastAPI and Vite services.
- Sign in as executive@example.test or manager@example.test using passwords chosen locally and never stored in the repository.
- Open SHP-24811 to inspect the synthetic 45 kg discrepancy, run validation, review source references, and observe export blockers.
- Open SHP-24812 to see unresolved comparisons when only an invoice is available.
- Run the backend default suite, the PostgreSQL integration suite, frontend tests, and the Vite production build.

Artifact	Truthful description
TDD.pdf	This technical design and implementation boundary document.
deployment.md	Windows-first local/demo setup with environment variable names only and synthetic login guidance.
Application	Source-controlled implementation; no credentials or client data included.
CSV	Backend-gated provisional response; not a claim to match an unavailable filing template.

End of document